Business Requirements Document (BRD) — Library Management System

1. Document control

Project name: Library Management System
Target users: Librarian, Students
Primary technology stack: FastAPI, PostgreSQL, REST APIs
Document type: Business Requirements Document (BRD)

2. Executive summary

The Library Management System is a practical, role-based application for a small academic library with an inventory of about 200 books. The system is intended to automate daily librarian operations while allowing students to discover books, check availability, join waitlists, and request new titles.[1][2][3]

The solution will be built with FastAPI as the backend framework and PostgreSQL as the relational database. The design must be clean, scalable, and API-first so it can support a future web or mobile frontend without reworking core business logic.[4][1][5]

The business focus is on real library behavior: multiple copies of the same book, fixed 7-day lending, automatic overdue fine calculation at ₹5 per day, queue-based waitlist allocation, and controls that prevent duplicate or conflicting issue transactions.[2][6][7]

3. Business context

The library currently manages a relatively small inventory, but day-to-day circulation can still become error-prone if book status, due dates, fines, and waiting queues are handled manually. A structured

system reduces dependency on manual registers or spreadsheets and gives the librarian a single source of truth for inventory and circulation status.[2][3]

Students also need limited but useful interaction with the system. They should be able to view available books, understand whether a book is currently issued, know the expected return date, join a waitlist when needed, and request books that are not in the collection.[2][6]

4. Business objectives

- Automate the core workflow of adding books, managing copies, issuing books, processing returns, tracking overdue loans, and collecting late fines.[2][3]
- Provide controlled self-service actions for students without exposing librarian-only administrative functions.[8][1]
- Prevent operational errors such as issuing an already issued copy, issuing the same title twice to the same student at the same time, or ignoring a pending waitlist.[9][6][7]
- Maintain accurate and queryable relational data for reporting, traceability, and future application growth.[4][5][3]

5. Project scope

5.1 In scope

- Authentication for librarian and student users.[8][1]
- Role-based authorization for each module and action.[8][10][11]
- Book master and copy-level inventory management.[2][3]
- Student book discovery with availability visibility.[2]
- Issue and return workflow with 7-day due date calculation.[2][9]
- Automatic fine calculation at ₹5 per overdue day, with no manual override in the core rule set.[6]
- Waitlist management for unavailable books with FIFO queue handling.[2][6][7]
- Automatic next-student allocation flow when a book is returned and a waitlist exists.[2][6]
- Student book requests and librarian approval or rejection.[2]
- RESTful API design across all modules.[4][5]

- PostgreSQL-based relational schema with constraints and referential integrity.[4][3]

5.2 Out of scope

- E-book borrowing and digital rights management.[2]
- Payments gateway integration for online fine payment.[2]
- Multi-branch library operations.[3]
- RFID, barcode scanner hardware integration, or biometric attendance in the first version.[1]
- Recommendation engines, AI search, or advanced academic integrations.[1]

6. Stakeholders

Stakeholder	Role	Interest
Librarian	Primary operational user	Manage inventory, issue/return, waitlists, fines, and student requests accurately.[1][2]
Students	Secondary users	View books, see availability, join waitlists, and request books.[2][6]
Library owner / institution admin	Sponsor / oversight	Ensure controlled operations, transparency, and manageable reporting.[3]
Development team	Delivery team	Implement a practical FastAPI + PostgreSQL solution with clear business rules.[4][1]

7. User roles

7.1 Librarian

The librarian is the primary user and owns all operational workflows. This role manages book records, copies, issue and return processing, fines, queue actions, and request approvals.[1][2]

7.2 Student

The student is a controlled-access user. This role can log in, view book information, check availability, view expected return dates for issued titles, join waitlists, and submit book requests, but cannot directly issue or return books in the core design.[2][6]

8. Current-state problem statement

In a manual or loosely managed setup, the librarian must track title details, individual copies, due dates, overdue cases, and student demand using separate notes or ad hoc processes. This often causes delays, duplicate records, and uncertainty about which specific copy is available or who should receive the next returned book.[2][3][7]

The absence of controlled validations also creates a risk that the same student may be given the same book again while an active loan already exists, or that a returned copy may be reissued without honoring the waitlist queue. A rules-driven system backed by relational constraints addresses these gaps.[9][6][7]

9. Future-state vision

The target state is a compact but reliable library operations platform where the librarian can complete all daily actions through a structured workflow and students can use limited self-service features through secure APIs. Every important activity should create consistent data records that reflect real-world library status at both the title level and the physical-copy level.[1][2][3]

The system should be small enough to implement quickly for a 200-book library but designed correctly enough to scale to larger volumes without redesigning the main entities or workflows.[4][5][3]

10. Business rules

1. The system shall support approximately 200 books at initial launch, but the architecture shall not hardcode this as a technical maximum.[1][3]

2. Each book title may have one or more physical copies, and each copy must be tracked individually.[2][3]
3. A loan period shall be exactly 7 days from issue date and time.[2][9]
4. A fine of ₹5 per overdue day shall be applied automatically for late returns, with no exception path in the standard workflow.[6]
5. A copy cannot be issued if its current status is issued, reserved, lost, or inactive.[2][7]
6. A student cannot hold more than one active loan for the same book title at the same time.[6]
7. If all copies of a title are unavailable, the student may join the waitlist for that title once only.[2][6]
8. Waitlist allocation shall follow FIFO order unless an authorized librarian explicitly cancels or skips an entry with a recorded reason.[6]
9. When a copy is returned and a waitlist exists, the system shall allocate the title to the next student in queue by creating a reservation/hold state rather than leaving it generally available.[2][6]
10. Overdue books not yet returned shall continue accumulating fines daily until return is processed.[9][6]
11. A student with an active loan for a title cannot be reissued another copy of the same title until the active loan is closed.[6]
12. All issue and return operations must be transaction-safe to prevent duplicate checkout of the same physical copy.[9][7]

11. Assumptions

- The system serves a single library location in the initial release.[3]
- Students are pre-registered or can be created through institutional onboarding handled by the system administrator or librarian.[1]
- The first release focuses on books only, not magazines or other media types.[2]
- The first release exposes APIs and may later receive a separate frontend UI without changing core backend logic.[4][5]

12. Functional requirements

12.1 Authentication and user management

Business need

The system requires secure login and role-based separation so that librarians can perform administrative actions while students can only access limited user flows.[8][1]

Requirements

- The system shall provide login for librarian and student users.[8][1]
- The system shall maintain user profiles with role assignment.[8][10]
- The system shall enforce RBAC on every protected API route.[8][10][11]
- The system shall allow activation and deactivation of user accounts.[1]
- The system shall record last login timestamp and audit fields for operational users.[1]

Key validations

- Username or email must be unique per user account.[1]
- Only active users may authenticate successfully.[8]
- Students cannot access librarian-only APIs.[8][11]

12.2 Book and inventory management

Business need

The librarian must manage title-level data and copy-level data separately because the library can hold multiple copies of the same book and each copy can have a different current status.[2][3]

Requirements

- The system shall allow the librarian to add, update, soft-delete, and view book titles.[1][3]
- The system shall store title details including ISBN, title, author, category, publisher, publication year, and optional description.[2]

[3]
- The system shall allow the librarian to create multiple copy records under a single book title.[2][3]
- The system shall track copy status as available, issued, reserved, lost, damaged, or inactive.[2][7]
- The system shall show total copies, available copies, issued copies, and reserved copies for each title.[2][3]

Key validations

- ISBN should be unique when provided, unless the organization intentionally allows missing ISBN for local records.[3]
- A copy record must belong to one valid book title.[3]
- A deleted title with active copies on loan must not be physically removed from the database.[3]

12.3 Book discovery

Business need

Students need a lightweight way to explore the library catalog and make informed decisions before contacting the librarian or joining a waitlist.[2]

Requirements

- The system shall allow students to view all active book titles.[2]
- The system shall allow search by title, author, category, and ISBN where available.[1][2]
- The system shall display title-level availability status.[2]
- The system shall display expected return date when all available copies are exhausted and at least one copy is currently issued.[2][3]
- The system shall display waitlist size for unavailable titles where business policy permits visibility.[6]

Key validations

- Inactive or deleted titles must not be visible to students.[1]
- Expected return date should be derived from active loan data, normally from the nearest due date among issued copies.[3]

12.4 Issue and return management

Business need

The librarian needs a reliable issue and return workflow that reflects physical circulation and automatically enforces due-date and fine rules.[2][9]

Requirements

- The system shall allow the librarian to issue an available copy to a student.[1][2]
- The system shall calculate due date automatically as issue date plus 7 days.[2][9]
- The system shall prevent issue if the student already has an active loan for the same title.[6]
- The system shall prevent issue if the selected copy is not available.[2][7]
- The system shall allow the librarian to process returns against the exact issued copy.[2][3]
- The system shall calculate final fine automatically at return time based on overdue days at ₹5 per day.[6]
- The system shall update copy status after return based on waitlist state, making it reserved for the next queued student or available if no waitlist exists.[2][6]
- The system shall preserve full loan history including issued_at, due_at, returned_at, overdue_days, and fine_amount.[9][3]

Key validations

- A loan can exist only for a valid student and a valid copy.[3]
- Only one active loan can exist per copy at a time.[7]
- Return date cannot be earlier than issue date.[3]
- Fine formula must be deterministic and non-editable in the base flow: fine = overdue_days × ₹5.[6]

12.5 Fine management

Business need

Late returns must create a simple, transparent, and predictable penalty so that circulation rules are enforced consistently.[2][6]

Requirements

- The system shall identify overdue loans automatically when current date exceeds due date and the item has not been returned.[9][3]
- The system shall calculate running overdue days for open overdue loans.[9]
- The system shall calculate fine accumulation at ₹5 per day until return is completed.[6]
- The system shall expose fine-related values in loan detail, student account view, and librarian operational lists.[9][6]

Key validations

- Fine must be zero if the item is returned on or before due date.[6]
- Overdue days must continue to increment for books not yet returned.[9]
- Fine amount must be stored at closure for audit while still being derivable from dates and rate.[9][3]

12.6 Waitlist management

Business need

When no copy is available, students need a fair queueing mechanism and librarians need controlled allocation when copies return.[2][7]

Requirements

- The system shall allow students to join a waitlist for a title that has no available copies.[2][6]
- The system shall maintain FIFO sequence using request timestamp or queue number.[6]
- The system shall prevent duplicate waitlist entries by the same student for the same title while the entry is active.[6]
- The system shall create a reservation/hold for the next student when a copy of that title is returned.[2][6]

- The system shall allow the librarian to view the queue for each title.[6]
- The system shall allow the librarian to cancel, skip, expire, or fulfill a waitlist entry with tracked reason and timestamp.[6]

Key validations

- A student with an active loan for the title cannot join the waitlist for the same title.[6]
- A student with an already active waitlist entry cannot join again.[6]
- Allocation must always use the earliest active waitlist entry first unless explicitly skipped by an authorized librarian.[6]

12.7 Book request system

Business need

Students should be able to signal demand for books that are missing from the library, and the librarian should manage these requests centrally.[2]

Requirements

- The system shall allow students to submit a new book request with title, author, optional ISBN, and remarks.[2]
- The system shall allow the librarian to review, approve, reject, or mark requests as procured.[2]
- The system shall link a procured request to a created book record where appropriate.[2]
- The system shall preserve request status history and decision remarks.[2]

Key validations

- A student cannot submit duplicate open requests for the same title-author combination without review.[2]
- Only librarians can approve or reject requests.[2]

13. Non-functional requirements

- The backend shall be implemented using FastAPI.[4][5]
- The database shall be PostgreSQL with a normalized relational design and enforced foreign keys.[4][3]
- The system shall expose RESTful APIs for all business modules.[4][5]
- The application architecture shall separate routing, business logic, validation schemas, and persistence layers to support future growth.[4][1]
- APIs shall be designed to support future web and mobile clients without changing core domain rules.[4][5]
- The system shall maintain auditability for creation, update, issue, return, reservation, and approval events.[1][3]
- The system shall handle concurrency safely so a copy cannot be checked out twice due to overlapping requests.[9][7]
- The system shall return meaningful validation errors for business-rule violations such as duplicate issue attempts or duplicate waitlist joins.[9][6]

14. Database schema requirements

14.1 Schema design principles

The schema must distinguish between a book title and a physical copy because availability exists at the copy level while discovery and waitlists often operate at the title level. The schema must also separate current circulation events from request and queue events so business rules remain clear and queryable.[2][3]

14.2 Required tables

Table	Purpose	Key columns
users	Stores login accounts	id, username/email, password_hash, role, is_active, created_at.[8][1]
students	Stores student profile data	id, user_id, student_code, full_name, department, status.[1]
librarians	Stores librarian profile data	id, user_id, employee_code, full_name, status.[1]
books	Stores title-level metadata	id, isbn, title, author, category, publisher, publication_year, is_active.[2][3]
book_copies	Stores physical copy records	id, book_id, copy_code/accession_no, status, condition, created_at.[2][3]
loans	Stores issue and return transactions	id, copy_id, student_id, issued_by, issued_at, due_at, returned_at, status, overdue_days, fine_amount.[9][3]
waitlist_entries	Stores per-title queue records	id, book_id, student_id, position or requested_at, status, allocated_copy_id, allocated_at.[2][6]
reservations	Stores hold/allocated records after waitlist processing	id, book_id, copy_id, student_id, waitlist_entry_id, status, reserved_at, expires_at.[6]
book_requests	Stores student requests for new books	id, student_id, title, author, isbn, remarks, status, reviewed_by, reviewed_at.[2]
fine_policies	Stores fine configuration if kept configurable	id, daily_rate, max_issue_days, active_flag.[9]

Table	Purpose	Key columns
audit_logs	Stores important system actions	id, actor_user_id, action, entity_type, entity_id, payload, created_at.[1]

14.3 Recommended relationships

- One users record belongs to exactly one role context, either librarian or student, depending on role.[8][1]
- One books record can have many book_copies.[2][3]
- One book_copy can have many historical loans, but only one active loan at a time.[3][7]
- One books record can have many waitlist_entries.[2][6]
- One student can have many loans, waitlist_entries, and book_requests.[2][3]
- One waitlist_entry may result in one reservation and then one eventual loan.[6]

14.4 Textual ER explanation

The ER model centers on books and book_copies. books represents the intellectual title, while book_copies represents each physical unit that can be issued separately.[2][3]

loans connects book_copies to students and captures the lifecycle of issue, due date, overdue calculation, and return. This design keeps circulation history intact and supports fine calculations and overdue reporting.[9][3]

waitlist_entries is linked to books, not book_copies, because students are waiting for the title regardless of which exact copy becomes available. Once a copy is returned, the system can create a reservation for the next student and associate the allocated copy at that point.[2][6]

book_requests is intentionally separate from books because a request may describe a book that is not yet part of the catalog. users, students, and librarians support authentication, role separation, and traceability.[8][1][2]

14.5 Constraints and validations at database level

- Primary keys on all master and transaction tables.[3]
- Foreign keys for all ownership and transactional relationships.[3]
- Unique constraint on `users.email` or `users.username`.[1]
- Unique constraint on `students.student_code` and `librarians.employee_code` where used.[1]
- Unique constraint on `book_copies.copy_code` or accession number.[2][3]
- Check constraint that `due_at >= issued_at`.[3]
- Check constraint that `returned_at IS NULL OR returned_at >= issued_at`.[3]
- Check constraint that `fine_amount >= 0` and `overdue_days >= 0`.[3]
- Partial unique rule recommended for one active loan per copy, implemented through application logic plus database strategy suited to PostgreSQL.[9][7]
- Unique active waitlist rule per student per book, enforced through application validation and database constraint strategy.[6]

15. API requirements

15.1 API design principles

The API set must be resource-oriented for CRUD operations and supplemented with action endpoints for domain actions such as issue, return, waitlist join, and approval workflows. This keeps the interface practical while preserving a RESTful structure.[4][1][5]

15.2 Authentication APIs

Method	Endpoint	Purpose
POST	/auth/login	Authenticate librarian or student and return token/session response.[8][1]
POST	/auth/logout	End authenticated session where applicable.[1]
GET	/auth/me	Fetch logged-in user profile and role.[8]

15.3 User and student APIs

Method	Endpoint	Purpose
GET	/students	List students, librarian only.[1]
POST	/students	Create student profile, librarian only.[1]
GET	/students/{student_id}	Get student detail and current status.[1]
PATCH	/students/{student_id}	Update student profile or status.[1]
GET	/students/{student_id}/loans	View student loan history and active loans.[9][3]

15.4 Book and copy APIs

Method	Endpoint	Purpose
GET	/books	List books for librarian or student view with filters.[1][2]
POST	/books	Create a new book title, librarian only.[1]
GET	/books/{book_id}	Get title detail, copy summary, and availability.[2]
PATCH	/books/{book_id}	Update title metadata, librarian only.[1]
DELETE	/books/{book_id}	Soft-delete or deactivate a book title, librarian only.[3]
POST	/books/{book_id}/copies	Add copy under a title, librarian only.[2][3]
GET	/books/{book_id}/copies	List all copies and statuses, librarian only or restricted view.[2]
PATCH	/copies/{copy_id}	Update copy status/condition, librarian only.[2]

15.5 Discovery APIs

Method	Endpoint	Purpose
GET	/catalog/books	Student-friendly book listing.[2]
GET	/catalog/books/{book_id}	Student-friendly title details with expected return date.[2][3]

15.6 Issue and return APIs

Method	Endpoint	Purpose
POST	/loans/issue	Issue an available copy to a student.[9]
POST	/loans/{loan_id}/return	Process return and final fine calculation.[9][6]
GET	/loans	List loans with filters for active, overdue, returned.[9][3]
GET	/loans/{loan_id}	Get loan detail.[9]
GET	/students/{student_id}/active-loans	View current open loans for a student.[9]
GET	/loans/overdue	Overdue operational list for librarian.[9][3]

15.7 Waitlist APIs

Method	Endpoint	Purpose
POST	/books/{book_id}/waitlist	Student joins waitlist for unavailable title.[2][6]
GET	/books/{book_id}/waitlist	Librarian views queue for a title.[6]
PATCH	/waitlist/{entry_id}/cancel	Cancel active waitlist entry.[6]
PATCH	/waitlist/{entry_id}/skip	Librarian skips current queue entry with reason.[6]
PATCH	/waitlist/{entry_id}/fulfill	Mark waitlist fulfilled after allocation/issue.[6]

15.8 Reservation APIs

Method	Endpoint	Purpose
GET	/reservations	View active holds/reservations.[6]
GET	/students/{student_id}/reservations	Student views current reserved books.[6]
PATCH	/reservations/{reservation_id}/expire	Expire hold if business process requires it.[6]

15.9 Book request APIs

Method	Endpoint	Purpose
POST	/book-requests	Student submits new book request.[2]
GET	/book-requests	Librarian views all requests.[2]
GET	/book-requests/{request_id}	Request details.[2]
PATCH	/book-requests/{request_id}/approve	Approve request, librarian only.[2]
PATCH	/book-requests/{request_id}/reject	Reject request, librarian only.[2]
PATCH	/book-requests/{request_id}/procure	Mark procured and optionally link new book record.[2]

15.10 Reporting APIs

Method	Endpoint	Purpose
GET	`/reports/dashboard`	Summary counts for books, copies, loans, overdue, waitlists, requests.[1][3]
GET	`/reports/fines`	Fine-related report output.[9]
GET	`/reports/popular-books`	Demand or issue frequency report for later phase.[3]

16. Validation rules

16.1 Loan validations

- Reject issue if selected copy is not `available`.[2][7]
- Reject issue if same student already has an active loan for the same title.[6]
- Reject issue if same loan is already returned.[9]
- Reject return if loan does not exist or is already closed.[9]

16.2 Waitlist validations

- Allow waitlist join only when no copies are currently available for the title.[2][6]
- Reject duplicate active waitlist by same student for same title.[6]
- Reject waitlist join if student already has active loan for the title.[6]

16.3 Book request validations

- Require title and requester identity.[2]
- Prevent repeated open requests by the same student for the same title-author pair.[2]

16.4 Data validations

- Title, author, and role fields must be non-empty where mandatory.[1][3]
- Dates must follow chronological integrity rules.[3]
- Fine values must be non-negative.[3]

17. Edge cases and expected behavior

17.1 Multiple students waiting for the same book

If several students are waiting for the same title, the system shall preserve queue order and allocate the next available returned copy to the first active waitlist entry in FIFO order. Later students remain in queue until subsequent copies are returned or the earlier entries are skipped or cancelled.[6][7]

17.2 Same student trying to issue multiple copies of the same book

If a student already holds an active loan for a title, the system shall reject any attempt to issue another copy of the same title to that student. This rule applies even if multiple physical copies exist.[6]

17.3 Fine accumulation over multiple days

For a book that is returned late, overdue days shall equal the number of days after the due date, and fine shall equal overdue days multiplied by ₹5. For a book that is still not returned, the system shall continue increasing overdue days and outstanding fine daily until closure.[9][6]

17.4 Handling overdue books not yet returned

Loans with passed due dates and no return timestamp shall appear in overdue lists and fine reports as active overdue items. These records remain open until the librarian processes the return.[9][3]

17.5 Prevent issuing if already issued to same user

If an active loan for the same title already exists for that student, the issue action shall fail with a clear business validation message rather than creating a duplicate transaction.[6]

17.6 Return when waitlist exists

When a returned copy belongs to a title with an active waitlist, the system shall not expose that copy as freely available first. It shall convert the copy into a reserved/held state for the next student in queue.[2][6]

17.7 Concurrent issue attempts on same copy

If two requests try to issue the same copy at nearly the same time, only one transaction shall succeed and the other must fail safely. This is required to avoid double checkout in concurrent environments.[9][7]

18. Reporting requirements

The librarian should have access to concise operational reports rather than complex BI features. Priority reports include active loans, overdue loans, total fines, available versus issued copies, waitlist volume by title, and pending book requests.[9][3]

Later analytics may include most-issued titles, average turnaround time, and fine trends, but these should remain secondary to the main circulation workflow in the first practical release.[3]

19. Security and audit requirements

- All protected APIs shall require authenticated access.[8][11]
- Role checks shall be enforced at route or service layer for each action.[8][10]
- Sensitive operations such as issue, return, request approval, and skip/cancel queue action shall be audit logged.[1]
- Passwords must never be stored in plain text and should be stored only as secure hashes.[8][1]

20. Development phases

Phase 1 — Authentication, book management, book viewing

- Login for librarian and student.[8][1]
- RBAC setup.[8][10]
- Book title CRUD and copy management.[1][2]
- Student catalog view with availability and expected return visibility.[2][3]

Phase 2 — Issue, return, and fine system

- Loan issue workflow.[9]
- Automatic 7-day due date.[2][9]
- Return processing.[9][3]
- Fine calculation at ₹5 per day and overdue reporting.[9][6]

Phase 3 — Waitlist and book requests

- Waitlist join and queue view.[2][6]
- Return-triggered allocation to next student.[2][6]
- Book request creation, approval, rejection, and procurement tracking.[2]

Phase 4 — Improvements

- Notifications for due dates, overdue items, and reservation availability.[2]
- Reports and lightweight analytics.[3]
- Operational refinements and usability improvements.[1]

21. Acceptance criteria

- The system supports librarian and student login with correct role enforcement.[8][1]
- The librarian can create books and manage multiple copies per title.[2][3]
- A book issue always creates a due date exactly 7 days after issue.[2][9]
- A late return always results in a fine of ₹5 per overdue day.[6]

- A student cannot issue another copy of the same title while an active loan already exists.[6]
- A student can join a waitlist only when no copies are available.[2][6]
- A returned copy is allocated to the next waitlisted student before becoming generally available.[2][6]
- Students can submit new book requests and librarians can approve or reject them.[2]
- The data model and APIs are implementable in FastAPI with PostgreSQL and can support future client applications.[4][5]

22. Recommended implementation notes

A practical implementation should use a layered FastAPI structure with routers, schemas, services, repositories, and database models so business rules stay testable and maintainable as the application grows.[4][1]

PostgreSQL should be used not only as storage but also to enforce core relational integrity through keys, constraints, and transactional updates. This is especially important for preventing duplicate active loans on a copy and maintaining clean queue behavior under concurrent requests.[9][3][7]